

# Introduction to Large Language Models (LLMs)

---

## 1. What Is Natural Language Processing (NLP)?

Natural Language Processing (NLP) is a field of Artificial Intelligence that focuses on enabling computers to understand, interpret, and generate human language.

### Examples of NLP Applications

- Machine Translation (English → French)
  - Text Classification (Spam vs Inbox)
  - Sentiment Analysis (Positive vs Negative)
  - Question Answering ("What is the capital of France?")
  - Chatbots (like ChatGPT)
- 

## 2. Understanding Language: Two Big Challenges

### 2.1 Syntax (Structure of Language)

Syntax is about how words are arranged to form correct sentences.

Example:

- ☒ "She saw the city." (Correct structure)
- ☐ "She city saw the." (Wrong structure)

Computers must learn sentence structure.

### 2.2 Semantics (Meaning of Language)

Semantics is about meaning.

Example:

- "Sherlock Holmes walked into the room."
- "Sherlock Holmes entered the room."

Different words, similar meaning.

Problem: Language can be ambiguous. Example: "I saw the man with a telescope." (Who has the telescope?)

---

## 3. Early Approach: Rule-Based Systems (Grammar)

We can define grammar rules to describe language structure.

Example Rules (Context-Free Grammar):

- $S \rightarrow NP VP$
- $NP \rightarrow N \mid D N$

- $VP \rightarrow V \mid V NP$

Example Sentence: "She saw the city"

Structure:

- $NP \rightarrow \text{She}$
- $VP \rightarrow \text{saw the city}$

Limitation: Writing rules for all English sentences is impossible.

---

## 4. Statistical Approach: Learning from Data

Instead of writing grammar rules manually, we let the computer learn patterns directly from text data.

The idea: More data  $\rightarrow$  Better pattern learning  $\rightarrow$  Better predictions.

---

### 4.1 Tokenization

Tokenization is the process of splitting text into smaller units (tokens).

Tokens can be:

- Words  $\rightarrow$  "I love AI"  $\rightarrow$  ["I", "love", "AI"]
- Characters  $\rightarrow$  ["I", " ", "I", "o", "v", "e"]
- Subwords (used in LLMs like GPT)

Why tokenization matters:

- Reduces text into manageable pieces
  - Allows counting and statistical analysis
  - Affects vocabulary size and performance
- 

### 4.2 n-grams

An n-gram is a sequence of **n consecutive words**.

General definition: For sentence with words:  $w_1, w_2, w_3, w_4 \dots$  An n-gram is:  $(w_i, w_{i+1}, \dots, w_{i+n-1})$

Example sentence: "I love machine learning"

Unigrams ( $n = 1$ ):

- I
- love
- machine
- learning

Bigrams ( $n = 2$ ):

- I love

- love machine
- machine learning

Trigrams ( $n = 3$ ):

- I love machine
- love machine learning

4-grams ( $n = 4$ ):

- I love machine learning
- 

## How n-grams are used

They are used to:

- Predict next word
- Build simple language models
- Measure text similarity
- Perform autocomplete

Example (Bigram model idea): If we see "machine", what word usually comes next? If "learning" appears most often after "machine", then the model predicts "learning".

---

## Which value of $n$ should we use?

There is NO single correct answer. It depends on the task.

$n = 1$  (Unigram)

- Very simple
- Ignores word order
- Works for sentiment analysis sometimes
- Not good for prediction

$n = 2$  (Bigram)

- Captures short dependencies
- Common in basic language models
- Good balance between simplicity and performance

$n = 3$  (Trigram)

- Captures more context
- Better predictions
- Requires more data

$n \geq 4$

- More context awareness
- Data becomes sparse

- Many combinations may never appear
- 

## Important Concept: Data Sparsity

As  $n$  increases:

- Number of possible combinations increases exponentially
- Many  $n$ -grams may never appear in training data

Example: If vocabulary = 10,000 words

Possible bigrams =  $10,000^2 = 100$  million Possible trigrams =  $10,000^3 = 1$  trillion

We usually do NOT have enough data for very large  $n$ .

---

## Practical Recommendation for Teaching

For beginners:

- Start with  $n = 2$  (bigram)
- Then show  $n = 3$  (trigram)
- Compare results

In modern LLMs:

- They do NOT explicitly use fixed  $n$ -grams
  - They use attention mechanisms to capture long-range dependencies
- 

## Classroom Demonstration Idea

Given sentence: "The cat sat on the"

What word comes next?

Using:

- Unigram → random common word
- Bigram → look at "the" → maybe "dog"
- Trigram → look at "on the" → maybe "mat"

Students will clearly see: Higher  $n$  → Better prediction (if enough data exists)

---

Used to predict next words and understand statistical structure of language.

---

## 5. Text Classification with Naive Bayes

### Goal

Classify text into categories (e.g., Positive / Negative, Spam / Ham).

---

## Simple Example

Training data:

Positive reviews:

- "I love this product"
- "This is amazing"

Negative reviews:

- "I hate this"
- "This is terrible"

We want to classify: "I love this"

---

### Step 1: Prior Probability

How many positive vs negative samples?

Positive = 2 Negative = 2 Total = 4

$P(\text{Positive}) = 2/4 = 0.5$   $P(\text{Negative}) = 2/4 = 0.5$

---

### Step 2: Word Probabilities

Vocabulary: [I, love, this, product, is, amazing, hate, terrible]

Now calculate:

$P(\text{love} \mid \text{Positive}) = (\text{\# of positive sentences containing "love"}) / (\text{total positive sentences}) = 1 / 2 = 0.5$

$P(\text{love} \mid \text{Negative}) = 0 / 2 = 0$

Already we see a problem: zero probability.

---

### Step 3: Naive Bayes Formula

We calculate:

$P(\text{Positive} \mid \text{"I love this"}) \propto P(\text{Positive}) \times P(\text{I} \mid \text{Positive}) \times P(\text{love} \mid \text{Positive}) \times P(\text{this} \mid \text{Positive})$

We multiply probabilities because of the **Naive assumption**: Words are independent given the class.

---

### Why "Naive"?

We assume:  $P(\text{love AND this} \mid \text{Positive}) = P(\text{love} \mid \text{Positive}) \times P(\text{this} \mid \text{Positive})$

In reality, words are NOT independent. But this simplification works surprisingly well.

---

## Zero Probability Problem

If a word never appears in Positive class:

$$P(\text{word}|\text{Positive}) = 0$$

Then the entire multiplication becomes 0.

That means the model becomes too confident.

---

## Solution: Laplace Smoothing

Add 1 to every count.

New formula:

$$P(\text{word} | \text{class}) = (\text{count}(\text{word}, \text{class}) + 1) / (\text{total\_words\_in\_class} + \text{vocabulary\_size})$$

Now even unseen words have small probability.

---

## Full Numeric Example

Suppose we want to classify: "love amazing product"

After counting and smoothing, assume:

$$P(\text{Positive}) = 0.5 \quad P(\text{love}|\text{Positive}) = 0.4 \quad P(\text{amazing}|\text{Positive}) = 0.3 \quad P(\text{product}|\text{Positive}) = 0.3$$

$$\text{Multiply: } 0.5 \times 0.4 \times 0.3 \times 0.3 = 0.018$$

$$\text{For Negative: } 0.5 \times 0.05 \times 0.05 \times 0.1 = 0.000125$$

Since  $0.018 > 0.000125$  Prediction = Positive

---

## Why Naive Bayes Works Well

- Very fast
  - Works well with small data
  - Good baseline model
  - Performs surprisingly well for spam detection
- 

## When It Fails

- Cannot capture word order
  - Cannot understand negation well Example: "not good" might be misclassified as positive
  - Assumes independence (not realistic)
- 

## Teaching Demonstration Idea

Ask students:

Which is positive?

1. "good"
2. "not good"

Naive Bayes may classify both as positive if "good" appears often in positive class.

This motivates more advanced models like neural networks and Transformers.

---

## 6. Word Representation: Converting Words to Numbers

Computers understand **numbers**, not words. So we must convert every word into a numerical vector.

The question is: *How do we represent meaning as numbers?*

---

### 6.1 One-Hot Encoding

The simplest way to represent a word is using a **one-hot vector**.

Step 1: Build a vocabulary.

Example vocabulary: ["he", "wrote", "a", "book"]

Step 2: Assign each word an index:

he → index 0 wrote → index 1 a → index 2 book → index 3

Step 3: Represent each word as a vector of zeros with a single 1 at its index.

he → [1, 0, 0, 0] wrote → [0, 1, 0, 0] a → [0, 0, 1, 0] book → [0, 0, 0, 1]

---

### Full Numerical Example

Sentence: "he wrote a book"

Representation as vectors:

he → [1, 0, 0, 0] wrote → [0, 1, 0, 0] a → [0, 0, 1, 0] book → [0, 0, 0, 1]

If vocabulary size = 10,000 words, then each vector has length 10,000.

Only ONE position is 1. All others are 0.

---

### Distance Problem Example

Suppose we add new words: ["book", "novel", "breakfast"]

One-hot vectors might look like:

book  $\rightarrow$  [1,0,0] novel  $\rightarrow$  [0,1,0] breakfast  $\rightarrow$  [0,0,1]

Now calculate similarity.

Distance(book, novel) = same as Distance(book, breakfast)

Because: They differ in exactly one position.

So the model cannot understand that:

book  $\approx$  novel book  $\neq$  breakfast

All words are equally far from each other.

---

### Code Example (One-Hot)

```
vocab = ["book", "novel", "breakfast"]
word_to_index = {w:i for i,w in enumerate(vocab)}

import numpy as np

def one_hot(word):
    vec = np.zeros(len(vocab))
    vec[word_to_index[word]] = 1
    return vec

print("book:", one_hot("book"))
print("novel:", one_hot("novel"))
print("breakfast:", one_hot("breakfast"))
```

---

### Problems with One-Hot Encoding

1. Very large vectors (size = vocabulary size)
2. Sparse (mostly zeros)
3. No semantic similarity
4. Cannot generalize meaning

This motivates a better representation: **Embeddings**.

---

## 7. Word Embeddings (Distributed Representation)

Instead of representing a word using a single 1 (one-hot), we represent each word using **many numbers**.

This is called a **distributed representation**.

Meaning is distributed across multiple dimensions of a vector.

---

## Intuition

In embeddings:

- Each word is a vector (for example 50, 100, or 300 numbers).
- Similar words have similar vectors.
- Distance between vectors represents similarity.

Example intuition:

book  $\approx$  novel king  $\approx$  queen breakfast  $\approx$  lunch

Unlike one-hot, similar words are mathematically close.

---

## Small Numeric Example

Imagine 2-dimensional embeddings (very simplified):

book  $\rightarrow$  [0.9, 0.8] novel  $\rightarrow$  [0.88, 0.79] breakfast  $\rightarrow$  [0.1, 0.95]

Now compare distances.

Distance(book, novel)  $\rightarrow$  very small Distance(book, breakfast)  $\rightarrow$  much larger

Now the model understands semantic similarity.

---

## Cosine Similarity (How we measure similarity)

We often use **cosine similarity** instead of Euclidean distance.

Formula:

$\text{cosine\_similarity}(A, B) = (A \cdot B) / (\|A\| \|B\|)$

Value range:

- 1  $\rightarrow$  very similar
  - 0  $\rightarrow$  unrelated
  - -1  $\rightarrow$  opposite
- 

## Code Example: Cosine Similarity

```
import numpy as np

book = np.array([0.9, 0.8])
novel = np.array([0.88, 0.79])
breakfast = np.array([0.1, 0.95])

def cosine_sim(a, b):
```

```
return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

print("book vs novel:", cosine_sim(book, novel))
print("book vs breakfast:", cosine_sim(book, breakfast))
```

Students will see: book–novel similarity is higher than book–breakfast.

---

## How Are Embeddings Learned?

Embeddings are learned from **context**.

Key idea (Distributional Hypothesis): "You shall know a word by the company it keeps." — J.R. Firth

If two words appear in similar contexts, their vectors become similar.

Example sentences:

- He wrote a book
- She wrote a novel

Because "book" and "novel" appear in similar positions, the model pushes their vectors closer together.

---

## Word2Vec (Core Idea)

Word2Vec trains a small neural network to:

- Predict a word from its context (CBOW)
- Predict context from a word (Skip-gram)

Over time: Vectors move in space to reduce prediction error.

As training improves, semantic structure emerges automatically.

---

## Famous Relationship Example

king – man + woman  $\approx$  queen

This means: The "gender" direction in vector space is learned.

The model captures relationships, not just similarity.

---

## Why Embeddings Are Powerful

1. Dense (small size, e.g., 100 numbers instead of 10,000)
  2. Capture semantic similarity
  3. Capture relationships
  4. Allow generalization to unseen combinations
-

## Limitation of Classic Word Embeddings

Traditional embeddings (Word2Vec, GloVe):

- Each word has only ONE vector
- "bank" (river) and "bank" (money) share the same embedding

Modern LLMs solve this using **contextual embeddings**: The meaning changes depending on the sentence.

Example:

- "I went to the bank to deposit money"
- "I sat by the river bank"

In Transformers, "bank" will have different vector representations in each sentence.

---

## 8. Neural Networks for Language

Neural networks are mathematical models that learn patterns from data.

In NLP, neural networks take **numerical word representations (embeddings)** as input and produce numerical outputs.

These outputs can represent:

- A category (spam / not spam)
  - The next word in a sentence
  - A translated sentence
  - An answer to a question
- 

### Basic Idea

Neural Network = Function approximator

It learns a function:

Input → Hidden layers → Output

For language tasks, the input is usually:

- A sequence of word vectors

The output could be:

- A probability distribution over vocabulary
  - A class label
  - Another sequence of word vectors
- 

### Simple Classification Example

Task: Sentiment Analysis

Sentence: "I love this movie"

Step 1: Convert each word to embedding vectors

I → [0.2, 0.1, ...] love → [0.9, 0.7, ...] this → [0.1, 0.2, ...] movie → [0.6, 0.5, ...]

Step 2: Combine vectors (for example by averaging)

Sentence vector = average of word vectors

Step 3: Pass into neural network

Output: [0.92, 0.08]

Meaning: 92% Positive 8% Negative

The network learns weights that detect patterns such as:

- "love" → positive signal
  - "hate" → negative signal
- 

## Sequence-to-Sequence Learning

Some tasks require generating a sequence as output.

Example: Machine Translation

Input (English): "The cat is small"

Output (French): "Le chat est petit"

Here both input and output are sequences.

This is called **Sequence-to-Sequence (Seq2Seq) learning**.

The model must:

1. Understand the full input sentence
  2. Generate output words one by one
- 

## Why Language Is Hard for Neural Networks

1. Sentences have variable length
2. Word order matters
3. Long-range dependencies exist Example: "The book that I bought yesterday is interesting" The word "book" connects to "is interesting"
4. Context changes meaning

Because of these challenges, special neural architectures were developed:

- Recurrent Neural Networks (RNNs)
- LSTMs / GRUs

- Transformers (modern LLMs)
- 

### Tiny Numerical Example (Conceptual)

Suppose we want to predict the next word.

Input: "I love"

The neural network produces probabilities:

$P(\text{movie}) = 0.6$   $P(\text{pizza}) = 0.3$   $P(\text{dogs}) = 0.1$

The model chooses the highest probability → "movie"

Output sentence: "I love movie"

(With more training, grammar improves.)

---

### Key Insight

Neural networks do NOT memorize grammar rules.

They learn statistical patterns by:

- Adjusting weights
- Minimizing prediction error
- Using gradient descent

Over millions or billions of examples, they learn deep representations of language.

This foundation leads to advanced architectures like Encoder–Decoder models and Transformers.

---

## 9. Encoder-Decoder Architecture

The Encoder–Decoder model was one of the first successful neural architectures for machine translation and sequence generation.

It is designed for **sequence-to-sequence problems**, where:

- Input = sequence (sentence)
- Output = another sequence (sentence)

Example: English → French translation Question → Answer generation Text → Summary

---

### Encoder

The encoder reads the input sentence **one word at a time**.

Example input: "What is the capital of France"

Process:

1. Convert each word to an embedding.
2. Pass embeddings through a Recurrent Neural Network (RNN/LSTM/GRU).
3. At each step, the model updates a hidden state.

Hidden state = compressed memory of everything seen so far.

After reading the full sentence, the encoder produces a **final hidden vector**.

This vector is supposed to represent the meaning of the entire sentence.

You can think of it as:

Sentence → Single summary vector

---

## Simple Numerical Intuition

Imagine each hidden state is a 4-dimensional vector:

After "What" → [0.2, 0.1, 0.5, 0.3] After "What is" → [0.4, 0.2, 0.6, 0.1] After full sentence → [0.9, 0.7, 0.2, 0.8]

The final vector contains encoded meaning.

---

## Decoder

The decoder takes the encoder's final hidden vector and generates output words **one by one**.

Example output: "The capital is Paris"

Steps:

1. Start with special token
2. Generate first word
3. Feed generated word back into decoder
4. Generate next word
5. Continue until token

At each step, the decoder produces a probability distribution over the vocabulary.

Example probabilities when predicting next word:

$P(\text{Paris}) = 0.72$   $P(\text{London}) = 0.10$   $P(\text{Berlin}) = 0.08$  ...

The highest probability word is selected.

---

## Why This Architecture Was Important

Before encoder-decoder models:

- Neural networks required fixed-size inputs and outputs.

- Could not easily handle variable-length sentences.

Encoder–Decoder solved this by:

- Compressing input into a vector
  - Generating output step-by-step
- 

## Main Limitation

Problem: The entire input sentence is compressed into ONE vector.

For short sentences, this works well.

For long sentences:

- Important information may be lost.
- Early words may be forgotten.
- Performance decreases as sentence length increases.

Example: "The book that I borrowed from the library last week after finishing my project is interesting"

The model must remember "book" until the end.

This compression bottleneck motivated the development of the **Attention mechanism**, which allows the decoder to look back at all encoder states instead of only one final vector.

---

## 10. Attention Mechanism

Attention allows the model to focus on important words.

Example: Question: "What is the capital of Massachusetts?" Model focuses on: "capital" and "Massachusetts"

Attention = Weighted importance of words.

---

## 11. Transformers (Modern LLM Architecture)

Transformers use:

- Self-Attention
- Positional Encoding
- Multi-Head Attention

Advantages:

- Parallel processing
- Better performance
- Handles long text better

Most modern LLMs (GPT, BERT) use Transformers.

---

## 12. What Is a Large Language Model (LLM)?

An LLM is:

- A very large Transformer-based neural network
- Trained on massive text data
- Learns patterns of language
- Can generate human-like text

Examples:

- ChatGPT
- GPT-4
- Claude
- Gemini

---

## 13. Coding Labs (Python)

---

### Lab 0 — Setup

```
pip install nltk scikit-learn numpy matplotlib
```

(Optional for embeddings)

```
pip install gensim
```

---

### Lab 1 — Tokenization (split text into words)

```
import re

def tokenize(text: str):
    # Lowercase to reduce duplicates ("Book" vs "book")
    text = text.lower()

    # Keep letters/numbers and split into tokens
    tokens = re.findall("[A-Za-z0-9_]+", text)

    return tokens

text = "I saw the man on the mountain with a telescope."
print(tokenize(text))
```

**What to explain while teaching**

- Lowercasing reduces vocabulary size.
- `findall` pulls out word-like pieces.
- Output is a list of tokens.

**Mini exercise:** Change the sentence and observe how tokens change.

---

## Lab 2 — n-grams (learn common word sequences)

```
from collections import Counter

def ngrams(tokens, n: int):
    # Create tuples of length n: (w1,w2,...,wn)
    return [tuple(tokens[i:i+n]) for i in range(len(tokens)-n+1)]

corpus = (
    "how often have i said to you that when you have eliminated the impossible "
    "whatever remains however improbable must be the truth"
)

tokens = tokenize(corpus)
trigrams = ngrams(tokens, 3)
counts = Counter(trigrams)

print(counts.most_common(10))
```

### What to explain

- An n-gram is a sliding window over tokens.
- Counts reveal common language patterns.

**Mini exercise:** Compare bigrams (n=2) vs trigrams (n=3).

---

## Lab 3 — Text Classification (Spam vs Ham) with Bag-of-Words + Naive Bayes

Pipeline:

1. Text → numbers (Bag-of-Words)
2. Train Naive Bayes
3. Predict new messages

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report

texts = [
    "Win money now",
```

```

    "Limited offer claim prize",
    "Hi, can we meet tomorrow?",
    "Please review the attached report",
    "Cheap pills available",
    "Lets schedule a project call"
]
labels = [1, 1, 0, 0, 1, 0] # 1=spam, 0=ham

X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.33, random_state=42
)

vectorizer = CountVectorizer(lowercase=True)
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

model = MultinomialNB(alpha=1.0) # alpha = Laplace smoothing
model.fit(X_train_vec, y_train)

pred = model.predict(X_test_vec)
print(classification_report(y_test, pred))

new_text = ["claim your free prize now"]
new_vec = vectorizer.transform(new_text)
print("Prediction (1=spam,0=ham):", model.predict(new_vec)[0])

```

### What to explain (teaching notes)

- `CountVectorizer` builds a vocabulary and turns each message into word counts.
- `MultinomialNB` works well with count data.
- `alpha=1.0` is Laplace smoothing (avoids zero probabilities).

**Mini exercise:** Add 10 new messages and see how results change.

---

## Lab 4 — One-Hot Encoding vs Embeddings

### 4A) One-hot (concept demo)

```

vocab = ["he", "wrote", "a", "book"]
word_to_index = {w: i for i, w in enumerate(vocab)}

def one_hot(word: str):
    vec = [0] * len(vocab)
    vec[word_to_index[word]] = 1
    return vec

print("he ->", one_hot("he"))
print("book ->", one_hot("book"))

```

### What to explain

- Vector length = vocabulary size.
- Similar words are not close in one-hot.

### 4B) Train small Word2Vec embeddings (optional)

```
# Optional: requires gensim
from gensim.models import Word2Vec

sentences = [
    tokenize("he wrote a book"),
    tokenize("she wrote a novel"),
    tokenize("he read a book"),
    tokenize("she read a novel"),
    tokenize("breakfast and lunch"),
    tokenize("lunch and dinner")
]

w2v = Word2Vec(sentences=sentences, vector_size=20, window=3, min_count=1, sg=1)

print("Most similar to 'book':", w2v.wv.most_similar("book", topn=5))
```

### What to explain

- Embeddings learn from context.
- Similar contexts → similar vectors.

---

## Lab 5 — Attention intuition (weights over words)

```
import numpy as np

words = ["what", "is", "the", "capital", "of", "massachusetts"]

# Pretend these are learned attention scores
scores = np.array([0.04, 0.02, 0.01, 0.28, 0.03, 0.62])

# Softmax to convert to probabilities
exp = np.exp(scores)
attn = exp / exp.sum()

for w, a in zip(words, attn):
    print(w, "->", round(float(a), 3))
```

### What to explain

- Attention says which input words matter most *right now*.
- Higher attention on "massachusetts" makes sense for predicting "Boston".

---

# Lab Assignment

---

Code Generation with HumanEval Dataset (Kaggle) + Similarity Check (Core Evaluation)

In this task, install **Llama 3** locally using Ollama, then use the model to generate Python solutions for **10 randomly selected HumanEval problems**. They will evaluate the results in two ways:

1. **Code similarity** to the reference solution — secondary

## Dataset

Use the Kaggle dataset:

- OpenAI HumanEval Code Gen dataset (164 Python problems) <https://www.kaggle.com/datasets/thedevastator/openai-humaneval-code-gen/data>

Each sample typically contains:

- **prompt** (function signature + docstring)
- **canonical\_solution** (reference solution)
- **test** (unit tests)
- **entry\_point** (function name)

## Part 1 — Install Llama 3 locally (Ollama)

1. Install Ollama
2. Pull the model:

```
ollama pull llama3.1:8b
```

3. Verify:

```
ollama run llama3.1:8b
```

## Part 2 — Select 10 problems

Students must select **10 problems** from the dataset.

Rules:

- Pick 10 different **task\_ids**
- Either random (recommended) or first 10 (if you want consistency)

Students must submit the list of chosen task IDs.

### Part 3 — Prompt the local LLM to generate code

For each selected problem, use the model to generate the function body.

Prompt template (recommended): "You are a Python coding assistant. Complete the function exactly as specified. Return **only** valid Python code. Do not add explanations. Do not change the function name or signature."

Input to the model = the dataset `prompt`.

Save output as:

- `llm_<task_id>.py`

### Part 4 — Similarity score vs reference (required)

Compute a similarity score between:

- `canonical_solution` (reference)
- the LLM solution

Use at least one method:

- `difflib.SequenceMatcher` ratio

Example:

```
import difflib

similarity = difflib.SequenceMatcher(None, ref_code, gen_code).ratio()
print(similarity)
```

Then discuss:

- A solution can be **high similarity but still wrong** (small bug)

### Deliverables

Students submit:

- Model name (`llama3.1:8b`) + machine specs
- The 10 chosen task IDs
- Prompts used
- LLM-generated code files
- A results table with:
  - `task_id`
  - `pass/fail`

- similarity score
- brief note (what went wrong if failed)

- 

